

Metriche del Software

Principali metriche del Software

Indice

◆ Metriche dimensionali

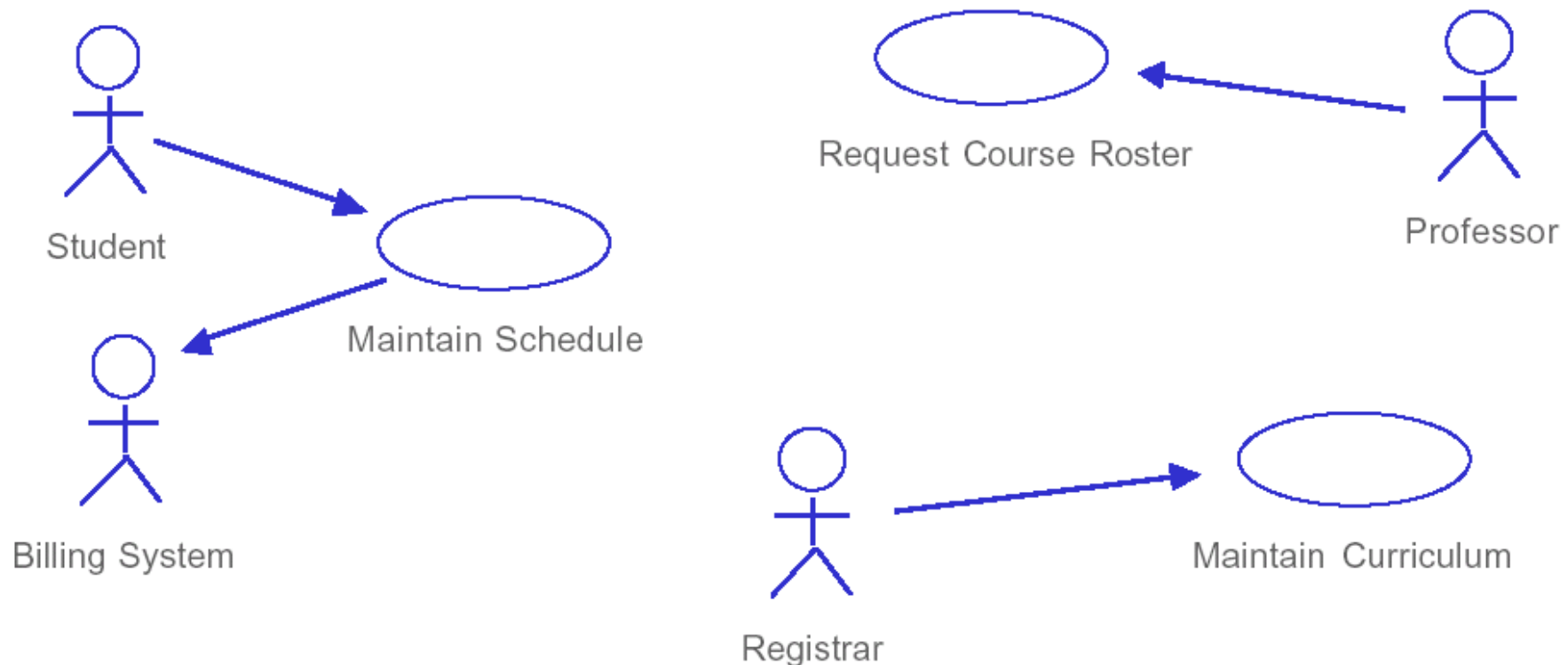
- Use case
- Function Points
- LOC

Metriche dimensionali

- ◆ La definizione delle metriche riguardanti le dimensioni del software dipende da diversi fattori, in particolare la fase dello sviluppo in cui si fa l'osservazione:
 - requisiti: numero di use case (in ambito object-oriented)
 - specifiche: Function Points
 - codifica: linee di codice (LOC), sono spesso indicate come “metriche dimensionali” per antonomasia

Metriche su use case

- ◆ Il numero di use case indica il numero di funzionalità percepibili dall'utente



Osservazioni

- ◆ Le funzionalità percepibili dall'utente sono rappresentabili a diversi livelli di granularità
 - *il numero cambia !*
- ◆ Catturano bene solo i requisiti funzionali
- ◆ In sostanza vanno bene nelle fasi alte, quando ci sono poche informazioni disponibili

I Punti Funzione

- ◆ I punti funzione vengono ideati nel 1975 da Allan Albrecht (IBM)
- ◆ L'idea fondamentale è di rappresentare le dimensioni di un prodotto software caratterizzato in base alle sue funzionalità, anziché rispetto alle sue dimensioni fisiche
- ◆ Definizioni seguenti:
 - IFPUG
 - Mark II Function Point
 - Feature point
 - Full Function Point
 - ...

La definizione originale

“una tecnica per validare le stime dello sforzo lavorativo necessario per progettare e sviluppare applicazioni software su misura”

Osservazioni iniziali

- ◆ Il “valore” di un prodotto software dipende dalle sue funzionalità
- ◆ La “funzionalità” è in relazione con gli input e gli output del sistema
- ◆ L’unità di misura deve essere comprensibile dall’utente e collegata ai requisiti da lui espressi
- ◆ Tale misura deve essere calcolabile nelle prime fasi dello sviluppo (quando si richiede la stima)
- ◆ Una misura di questo tipo è utilizzabile per valutare la “funzionalità rilasciata all’utente”

Gli obiettivi fondamentali

- ◆ Indipendenza dalla tecnologia
 - ci si basa sui documenti di specifica
 - realizzazioni diverse (per linguaggio di programmazione o per strumenti di sviluppo) danno la stessa misura
- ◆ Misurare tutte le funzionalità fornite all'utente
- ◆ Misurare solo le funzionalità fornite all'utente

Funzioni elementari (Function Types)

- ◆ Il metodo di Albrecht prevede di identificare e contare il numero di:
 - Input
 - Output
 - Archivi logici
 - Archivi di interfaccia
 - Inquiry esterne
- ◆ Questi elementi dovrebbero rappresentare nel loro complesso la “manifestazione esterna” di qualunque applicazione, cioè ne rappresentano le funzionalità.

Conteggio delle funzionalità di “gestione dati”

- ◆ Archivio logico interno
 - un insieme omogeneo di dati o di informazioni di controllo usati e gestiti dall'applicazione
- ◆ Archivio logico di interfaccia
 - un insieme omogeneo di dati o di informazioni di controllo usati dall'applicazione ma generati e mantenuti da altre applicazioni

Conteggio delle funzionalità di “gestione funzioni”

◆ Input esterno

- operazione elementare di elaborazione di un dato o informazione di controllo proveniente dall'esterno dell'applicazione

◆ Output esterno

- operazione elementare che genera un dato o un'informazione di controllo verso l'esterno dell'applicazione

◆ Inquiry esterna

- operazione elementare che coinvolge input e output

Procedimento di definizione

- ◆ Albrecht ha considerato 24 applicazioni
 - 18 COBOL, 4 PL/1, 2 DMS
 - dalle 3000 alle 318000 LOC
 - da 500 a 105200 ore-persona
- ◆ Ha definito una formula per il calcolo di FP in base ai function types, e una per il calcolo dello sforzo in base ad FP, cercando di minimizzare gli errori per l'insieme delle 24 applicazioni
- ◆ Risultato: errori del 24% sulle stime dello sforzo

Applicazione

- ◆ Al termine della progettazione degli aspetti esteriori dell'applicazione
- ◆ All'installazione e quindi periodicamente (per valutare la produttività di sviluppo e manutenzione)
- ◆ Calcolo in base ai Function Types, opportunamente pesati
- ◆ *Correzione finale (da -35% a +35%) in base a 14 parametri*
- ◆ Il numero finale è utilizzabile come variabile indipendente per il calcolo di diverse grandezze (come lo sforzo di sviluppo).

Tabella dei pesi

- ◆ Si utilizza una tabella in cui appaiano alcuni pesi per i quali devono essere moltiplicati i diversi valori dei Function Types (la complessità è valutata in base alle caratteristiche dell'applicazione).

Function Types	Pesi		
	Semplice	Medio	Complesso
N. Inputs	3	4	6
N. Outputs	4	5	7
N. Richieste	3	4	6
N. Files	7	10	15
N. Int. esterne	5	7	10

Formula per il calcolo di FP

◆ FP è dato dalla seguente formula:

$$FP = FP \text{ grezzi} \times \left[0.65 + 0.01 \times \sum_{i=1}^{14} F_i \right]$$

Parametri correttivi F_i

0	1	2	3	4	5
Ininfluente	Incidenza scarsa	Incidenza moderata	Incidenza media	Incidenza significativa	Incidenza essenziale

Il sistema richiede procedure di recovery e backup affidabili?

È richiesta la trasmissione di dati?

Vi sono funzionalità che richiedono elaborazioni distribuite?

Le prestazioni sono critiche?

Il programma funzionerà in un ambiente operativo già pesantemente utilizzato?

Il sistema richiede funzionalità avanzate per l'immissione e la consultazione in linea dei dati?

Le funzionalità di immissione dei dati devono essere costruite tramite interfacce a finestre?

Gli archivi principali sono aggiornati in tempo reale?

Le informazioni scambiate tra utente e programma sono complesse?

Il codice del programma è complesso?

Il codice è scritto per essere riusabile?

Nel progetto sono include anche le attività di installazione e conversione?

Il programma è stato progettato per essere installato presso diversi utenti?

Il programma è stato progettato per facilitare un facile uso e la possibilità di apportare modifiche da parte dell'utente?

L'International Function Point Users Group

- ◆ Produce un manuale con le istruzioni per il conteggio dei FP
- ◆ Missione: curare l'evoluzione della tecnica FP
- ◆ Scopo delle varie versioni:
 - applicabilità fuori dal contesto originale
 - consistenza delle indicazioni tra organizzazioni diverse
 - esempi, applicazioni con GUI

Critiche ai function points

- ◆ Rispetto alla teoria rappresentazionale i FP non spiegano la correlazione tra la misura dei FP e la conoscenza empirica del concetto di funzionalità.
Esempi:
 - 6 file logici di bassa complessità, 6 interfacce di media complessità e 7 input di alta complessità contribuiscono in egual misura alla “funzionalità” del sistema: perché?
- ◆ i FP ci dicono che una applicazione è 1,7 volte più complessa di un'altra: come si spiega questo in termini concettuali? perché 1,7 e non 1,5 o 2?

Critiche ai Function Point

- ◆ La correzione dei FP grezzi appare irrazionale, essendo un tentativo di adattare la misura della funzionalità allo scopo di servire alla predizione dei costi di sviluppo.
 - Fenton: è come correggere la misura dell'altezza di una persona per poterla correlare alla misura dell'intelligenza.
 - La correzione non migliora la precisione delle stime, anzi appare che il semplice numero di input e output sia uno stimatore equivalente (se non migliore) dei Punti Funzione

Critiche ai Function Points

- ◆ Nel calcolo dei FP a partire dai Function Types si perde informazione. Si è verificato che utilizzando direttamente i Function Types come variabili indipendenti si ottiene un modello migliore
 - in pratica l'intervallo di variabilità delle stime diventa la metà rispetto al modello originale di Albrecht

Critiche ai Function Points

- ◆ Il conteggio dei punti funzione avviene nelle fasi iniziali del progetto, sulla base di documenti di specifica o di progetto, spesso del tutto informali, scritti in linguaggio naturale, con al più diagrammi.
- ◆ Quindi il conteggio avviene su basi piuttosto fragili, con un ampio margine di discrezionalità
 - Le specifiche vengono “interpretate” dall’esperto di conteggio

Critiche ai Function Points

- ◆ Le modalità di conteggio dovrebbero essere invece espresse in modo preciso, basandosi su documenti formali
 - oggi il conteggio è “un’arte”, di cui esistono esemplificazioni, ma nessuna definizione formale
 - i tool danno una mano, ma non automatizzano il processo
- ◆ Esperimenti hanno mostrato una ampia variabilità di risultati per conteggi effettuati da esperti sullo stesso documento
 - fino al 500% (anche per progetti piccoli)
 - mediamente del 30% per analisti di una stessa organizzazione

Critiche ai Function Points

- ◆ Il conteggio dipende dal contenuto delle specifiche
 - più sono dettagliate e più il numero di FP cresce
- ◆ Il conteggio è svolto con criteri ampiamente soggettivi.
- ◆ L'IFPUG ha anche lo scopo di rendere quanto più omogenei possibili i criteri di conteggio
 - cosa che appare comunque difficile, vista la mancanza di formalità nella procedura

Un esempio di calcolo di FP

- ◆ Consideriamo il problema di contare i FP di una tradizionale applicazione di gestione ordini, interattiva, scritta in Visual basic.

The screenshot shows a complex Visual Basic application window titled "Order File: 00101-000000". The interface is divided into several sections:

- Customer Information:** Includes fields for Name (James B. Chipman), Address (1234 BL. Industrial Street), and Telephone (123-456-7890). Below this is a table with columns "Telephone" and "Description" containing two entries.
- Order Details:** Includes fields for Order Number (1234), Order Date (01/01/01), and Order Status (New). Below this is a table with columns "Telephone" and "Description" containing two entries.
- Shipping Information:** Includes fields for Ship To (1234 BL. Industrial Street), Ship From (1234 BL. Industrial Street), and Ship Date (01/01/01). Below this is a table with columns "Telephone" and "Description" containing two entries.
- Order Summary:** Includes fields for Order Total (1234.56), Order Tax (123.45), and Order Net (1111.11). Below this is a table with columns "Telephone" and "Description" containing two entries.
- Order History:** Includes a list of order numbers and dates.

The application is written in Visual Basic and is used for managing orders.

File interni ed esterni

- ◆ Identifichiamo i gruppi logici di informazioni (file) usati o mantenuti (modificati) dall'applicazione.
 - Ipotizziamo che non ci siano altri file che quelli rappresentati direttamente nella schermata vista.
- ◆ I dati sullo schermo derivano da due file logici
 - il file degli ordini - modificato dall'applicazione
 - il file degli agenti - solo consultato, per lettura e validazione
- ◆ Abbiamo un Internal Logical File (ILF): il file degli ordini e un External Interface File (EIF): il file degli agenti.

Complessità dei file: definizione

- ◆ La complessità di un file logico dipende da due fattori:
 - il numero di record element types (RETs)
 - il numero di data element types (DETs).
- ◆ Nel nostro caso ciascun file ha un solo RET
 - avremmo più RET se ad es. ci fossero diversi tipi di ordini caratterizzati da diverse informazioni.
- ◆ I DET sono essenzialmente i “campi” dati contenuti in ciascun “record”
 - le ripetizioni di istanze (es. più numeri di telefono) non contano
 - se l'applicazione utilizzasse un DB relazionale dovremmo contare gli attributi delle tabelle che logicamente compongono il “file”

Complessità dei file di input e output

- ◆ Il conteggio di RET e DET dà il seguente risultato:

File Name	File Type	RETs	DETs
Order File	ILF	1	70
Agent File	EIF	1	5

Tabella per il calcolo della complessità

- ◆ Il calcolo della complessità si basa sulla tabella seguente:

RETs	DETs		
	1-19	20-50	50+
1	Low	Low	Average
2-5	Low	Average	High
6+	Average	High	High

Complessità dei file di I/O

- ◆ L'applicazione della tabella ai nostri dati dà il seguente risultato:

File Name	File Type	RETs	DETs	Complexity
Order File	ILF	1	70	Average
Agent File	EIF	1	5	Low

Input, output e interrogazioni

- ◆ Il passo successivo riguarda il conteggio del numero di input, output e interrogazioni associate alla schermata.
 - all'atto della presentazione della schermata si ha una interrogazione dei file per estrarre i dati da mostrare/editare
 - quando si preme OK si ha una transazione che aggiorna i dati del file degli ordini
- ◆ La complessità di una interrogazione/transazione si determina in base a
 - numero di logical file types referenced (FTRs)
 - numero di data element types (DETs).

Input, output e interrogazioni

◆ Nel nostro caso:

- per l'interrogazione iniziale 2 FTRs (numero di logical file types referenced), 73 DETs.
- Per la transazione di aggiornamento: 1 FTR (ordini), 68 DETs + l'evento di controllo (tasto return o click su OK) = 69 DETs.

Altre interrogazioni/transazioni

- ◆ La cancellazione di ordini (se possibile) sarebbe una ulteriore transazione.
- ◆ I messaggi di errore associati ad un input sono conteggiati come output separati.
- ◆ Nel nostro caso si ha un messaggio d'errore se il codice dell'agente non è valido o se gli indirizzi sono lasciati vuoti.
 - Si conta un FTR corrispondente al file degli agenti, consultato per validazione.
 - Si conta un DET per ogni messaggio: totale tre (codice non valido, indirizzo destinazione indefinito, indirizzo origine indefinito).

Tabelle per il calcolo della complessità

- ◆ La complessità degli input si calcola in base alla seguente tabella:

FTRs	DETs		
	1-4	5-15	16+
0-1	Low	Low	Average
2	Low	Average	High
3+	Average	High	High

Tabelle per il calcolo della complessità

- ◆ La complessità degli output si calcola in base alla seguente tabella:

FTRs	DETs		
	1-5	6-19	20+
0-1	Low	Low	Average
2-3	Low	Average	High
4+	Average	High	High

Complessità di interrogazioni, input e output

- ◆ La complessità delle transazioni si calcola usando la tabella del più complesso tra input ed output (di solito è l'output).
- ◆ Nel nostro caso la valutazione finale di input, output e transazioni è:

Transaction	Type	FTRs	DETs	Complexity
Screen Display	Inquiry	2	73	High
Edit	Input	1	69	Average
Error Messages	Output	1	3	Low

Calcolo dei FP

- ◆ Il calcolo finale dei FP avviene inserendo i dati trovati nella apposita tabella per la “pesatura”:

Element	Low	Average	High	Total
Input	0 x 3	1 x 4	0 x 6	4
Output	1 x 4	0 x 5	0 x 7	4
Inquiry	0 x 3	0 x 4	1 x 6	6
ILF	0 x 7	1 x 10	0 x 15	10
EIF	1 x 5	0 x 7	0 x 10	5
FP				29

- ◆ La nostra applicazione consta di 29 function points.

Metriche dimensionali

- ◆ Si basano su una misura diretta delle linee di codice del programma.
- ◆ Si usano diverse sigle che tuttavia fanno sostanzialmente riferimento allo stesso concetto:
 - LOC (lines of code) o SLOC (source LOC)
 - DSLOC (delivered SLOC)
 - DSI (delivered source instructions) o KDSI (migliaia (Kilo) DSI)

Metriche dimensionali: calcolo

- ◆ A seconda dell'utilizzo che si intende fare di queste metriche può aver senso escludere dal conteggio delle parti:
 - se voglio misurare quanto sviluppo escludo le istruzioni riusate da altri programmi senza modifiche
 - se voglio correlare le dimensioni con gli errori escludo le linee di commento (che non generano errori)
 - viceversa se sto valutando la manutenibilità del prodotto le linee di commento sono rilevanti
 - ...

Metriche derivate

- ◆ Produttività = $\text{LOC} / \text{mesi-uomo}$
- ◆ Qualità = $\text{Numero totale di errori} / \text{LOC}$
- ◆ Costo unitario = $\text{Costo totale} / \text{LOC}$
- ◆ Livello di documentazione = $\text{Pagine di documentazione} / \text{LOC}$

Valutazione: Metriche Dimensionali

◆ Pro

- Sono facilmente definibili.
- È facile misurare il valore di LOC per un qualsiasi programma.
- Sono basate su concetti molto diffusi tra gli addetti del settore.
- Sono utilizzate in molti metodi esistenti per la stima dei costi e per la misura della produttività.
- Quasi tutti i dati storici raccolti in passato sono basati su questo tipo di metriche.

Valutazione: Metriche Dimensionali

◆ Contro

- La dimensione espressa in linee di codice sorgente risulta fortemente dipendente da attitudine e capacità del programmatore
- La dimensione ha un “peso” diverso per linguaggi di programmazione diversi.
- LOC_{Ada} non confrontabili con LOC_C , ecc.
- Occorre definire in modo preciso cos'è una linea di codice.
 - » Si possono usare strumenti di conteggio (usare sempre lo stesso tool garantisce consistenza).
- Poiché gli indici di qualità vengono a dipendere dal numero di linee rilasciate, un tentativo di migliorare la produttività potrebbe portare chi sviluppa a scrivere più codice, senza una particolare attenzione alla qualità di ciò che si produce.

Valutazione: Metriche Funzionali

◆ Pro

- Sostanzialmente indipendente dalle capacità del programmatore (fa riferimento alle specifiche) e dal linguaggio di programmazione.

◆ Contro

- Si basano su una definizione di complessità molto semplificata.
- Il conteggio è parzialmente soggettivo. Non esiste un criterio univoco per i conteggi, le assegnazioni dei pesi e la valutazione dei parametri correttivi.
 - » Persone diverse danno generalmente valutazioni diverse.
 - » IFPUG cerca di fornire un criterio omogeneo.

Relazione tra metriche dimensionali e funzionali

- ◆ Secondo Albrecht e Gaffney esiste una correlazione tra metriche funzionali e dimensionali
 - determinata in base all'esame di 24 progetti reali.
- ◆ Valutazioni più recenti (Caper Jones):

Linguaggio	LOC/FP
Macro Assembler	213
C	150
Fortran 77	105
ANSI COBOL 85	91
Ada	71
Prolog, Lisp	64
Pseudo-language	53
C++, altri linguaggi OO	27
DB query language	13
Spreadsheet	6
Linguaggi 5a gen, VB	4



Complessità

- ◆ La dimensione del software è la metrica statica più evidentemente utile per caratterizzare un sistema SW.
- ◆ Tuttavia non riesce da sola a catturare tutti gli aspetti del sistema: come si comporta un programma grosso ma semplice rispetto a uno piccolo ma complesso?
- ◆ Un'altra metrica estremamente importante è la *complessità* del codice.
- ◆ Solo valutando sia dimensioni che complessità si ottiene un “identikit” del software sufficientemente completo

Valutazione della Complessità

◆ Difficile. Opzioni:

- valutazione soggettiva (ad esempio basata su interviste)
 - » importante avere un criterio uniforme di riferimento
- valutazione oggettiva
 - » ci vuole un algoritmo di calcolo, basato sugli attributi oggettivi del programma
- valutazione ibrida

Cyclomatic Branch

- ◆ Definita da Thomas McCabe nel 1976.
 - Rappresenta il numero di percorsi lineari indipendenti in un modulo.
- ◆ Terminologia:
 - Cyclomatic complexity,
 - complexity,
 - McCabe's complexity

Il Control flow

- ◆ L'esecuzione di un programma varia in dipendenza dei valori assunti dalle sue variabili di input e del flusso di controllo definito dalle strutture con predicati

```
procedure Quadrato;  
  var x, y, n: integer;  
  begin  
1.  read(x);  
2.  if x > 0  
    then begin  
3.      n := 1;  
4.      y := 1;  
5.      while x > 1 do  
        begin  
6.          n := n + 2;  
7.          y := y + n;  
8.          x := x - 1;  
        end;  
9.      write(y);  
    end;  
  end;  
end;
```

x=0

```
procedure Quadrato;  
  var x, y, n: integer;  
  begin  
1.  read(x);  
2.  if x > 0  
    then begin  
3.      n := 1;  
4.      y := 1;  
5.      while x > 1 do  
        begin  
6.          n := n + 2;  
7.          y := y + n;  
8.          x := x - 1;  
        end;  
9.      write(y);  
    end;  
  end;  
end;
```

x=1

```
procedure Quadrato;  
  var x, y, n: integer;  
  begin  
1.  read(x);  
2.  if x > 0  
    then begin  
3.      n := 1;  
4.      y := 1;  
5.      while x > 1 do  
        begin  
6.          n := n + 2;  
7.          y := y + n;  
8.          x := x - 1;  
        end;  
9.      write(y);  
    end;  
  end;  
end;
```

x>1

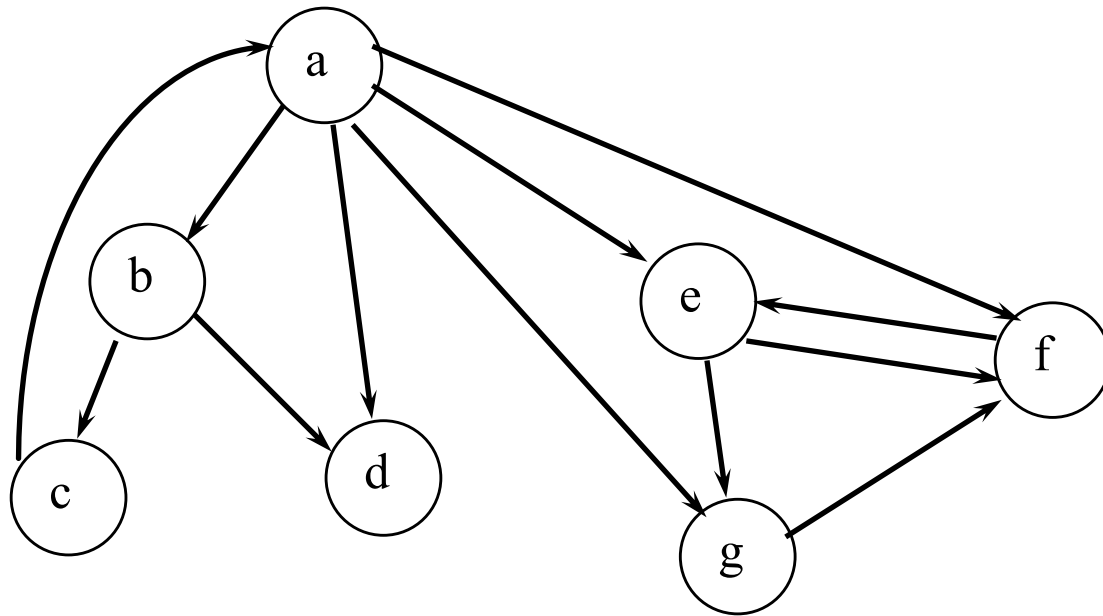
```
procedure Quadrato;  
  var x, y, n: integer;  
  begin  
1.  read(x);  
2.  if x > 0  
    then begin  
3.      n := 1;  
4.      y := 1;  
5.      while x > 1 do  
        begin  
6.          n := n + 2;  
7.          y := y + n;  
8.          x := x - 1;  
        end;  
9.      write(y);  
    end;  
  end;  
end;
```

Cyclomatic complexity

- ◆ La Complessità Ciclomatica è una metrica *strutturale* – *procedurale*, per stimare la "*difficoltà*" di un blocco di codice
- ◆ **Idea:** contare il numero di **punti di decisione** (if, for, while, etc...) presenti in un programma
- ◆ L'assunto è che quanti più sono tali punti, tanto più complesso (e peggiore) sarà il codice
 - Minore coesione
 - Peggiore leggibilità
 - Più error-prone
- ◆ Si basa sul concetto di *Grafo del Flusso di Controllo (GFC)*

Grafi e modelli di rappresentazione dei programmi

- Un grafo diretto (o orientato) è una coppia (N, E) dove N è un insieme di nodi e $E \subseteq N \times N$ è un insieme di archi



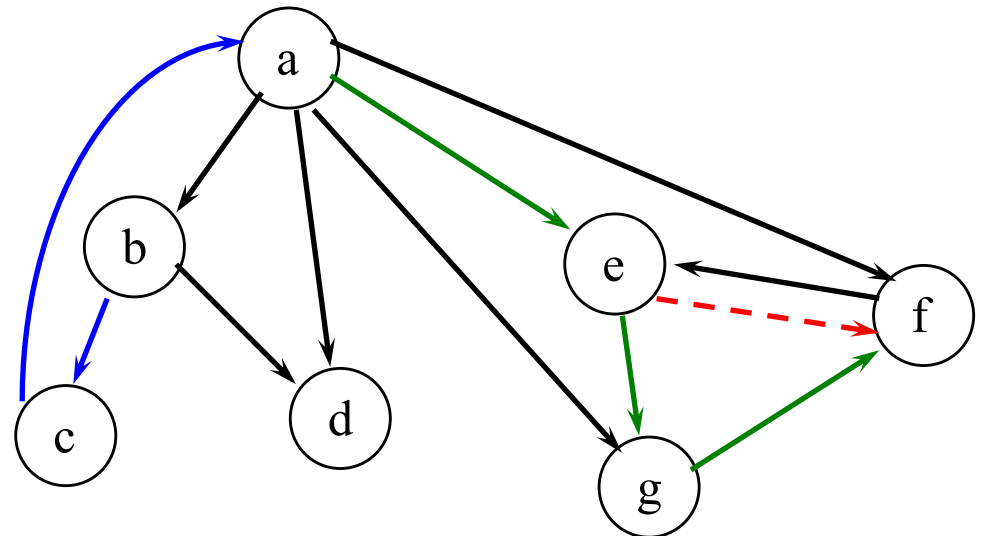
$N = \{ a, b, c, d, e, f, g \}$

$E = \{(a, b), (a, d), (a, e), (a, f), (a, g), (b, c), (b, d), (c, a), (e, f), (e, g), (f, e), (g, f)\}$

NB: gli archi del grafo possono essere etichettati ...

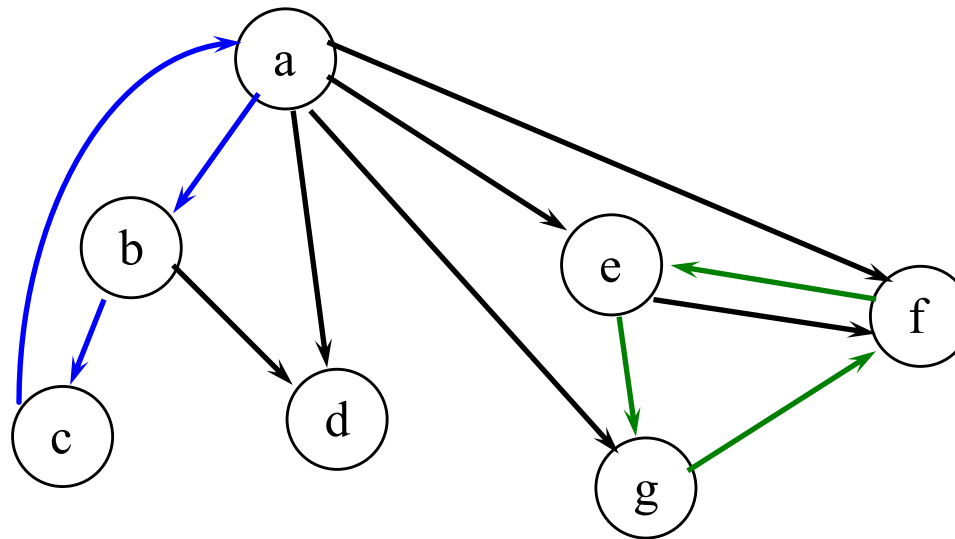
Cammini

- Per ogni arco $(n, m) \in E$, n è detto un *predecessore* di m e m è detto un *successore* di n . Un nodo può avere zero o più successori e zero o più predecessori
- Un cammino di lunghezza $k-1$ è una sequenza di nodi $\langle n_1, n_2, \dots, n_k \rangle$ tali che $\forall i, 1 \leq i \leq k-1, (n_i, n_{i+1}) \in E$
- **Esempio:**
 - $\langle a, e, g, f \rangle$ e $\langle b, c, a \rangle$ sono cammini
 - $\langle e, f, g \rangle$ non è un cammino



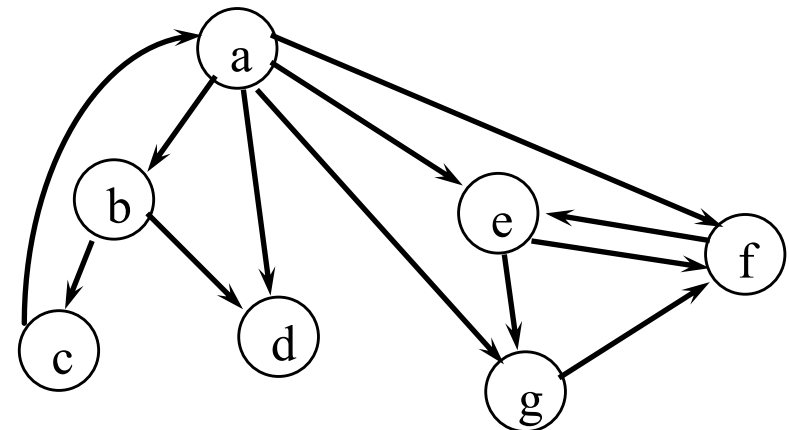
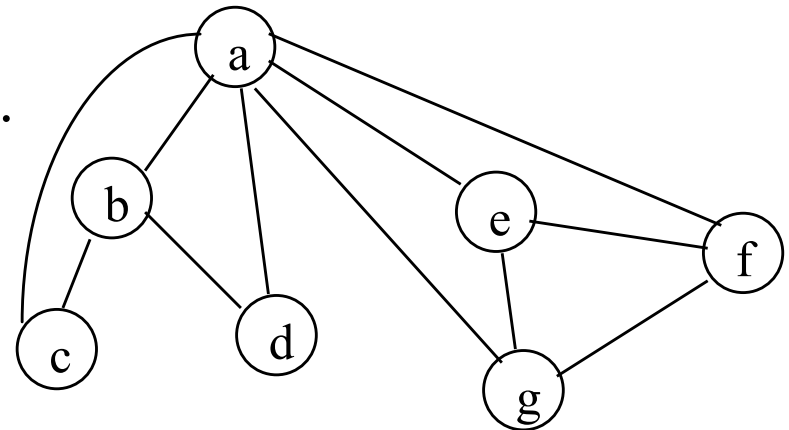
Circuiti (o cicli)

- Un circuito è un cammino $\langle \mathbf{n}_1, n_2, \dots, \mathbf{n}_k \rangle$ tale che $n_k = n_1$
- **Esempio:**
 - $\langle a, b, c, a \rangle$ e $\langle e, g, f, e \rangle$ sono circuiti
 - $\langle e, f, g, e \rangle$ non è un circuito



Raggiungibilità e grafi non orientati

- Un grafo orientato si dice **connesso** se ogni suo nodo è raggiungibile da ogni altro nodo sul grafo che si ottiene eliminando l'orientamento degli archi
- *Il grafo dei nostri esempi è connesso ...*
- Un grafo orientato si dice **fortemente connesso** se ogni nodo è raggiungibile da ogni altro nodo (ossia se esiste un ciclo che coinvolge tutti i nodi del grafo).
- *Il nostro grafo non è fortemente connesso ...*



Un modello di rappresentazione dei programmi

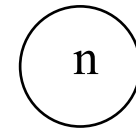
- Il *Grafo del Flusso di Controllo* di un programma P è una quadrupla $GFC(P) = (N, E, n_i, n_f)$ dove:
- (N, E) è un grafo diretto con archi etichettati
 $n_i \in N, n_f \in N, N - \{n_i, n_f\} = N_s \cup N_p$
- N_s e N_p sono insiemi disgiunti di nodi, ossia $N_s \cap N_p = \emptyset$
che rappresentano rispettivamente istruzioni e predicati
 $E \subseteq (N - n_f) \times (N - n_i) \times \{true, false, uncond\}$
rappresenta la relazione di flusso di controllo

Un modello di rappresentazione dei programmi

- ◆ n_i ed n_f sono detti *nodo iniziale* e *nodo finale*
- ◆ Un nodo in $N_s \cup \{n_i\}$ ha un solo successore immediato ed il suo arco è etichettato con *uncond*
- ◆ Un nodo in N_p ha due successori immediati e i suoi archi uscenti sono etichettati rispettivamente con *true* e *false*
- ◆ Un GFC(P) è ben formato se esiste un cammino dal nodo iniziale n_i ad ogni nodo in $N - \{n_i\}$ e da ogni nodo in $N - \{n_f\}$ al nodo finale n_f
- ◆ Diremo semplicemente cammino o cammino totale un cammino da n_i a n_f

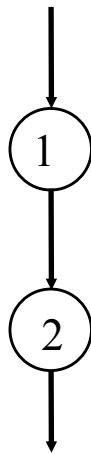
Costruzione del control flow graph per programmi strutturati

- Il grafo è costruito secondo la seguente notazione:
- **NODO** rappresenta un'istruzione, identificato da un numero n
- **ARCO** rappresenta il passaggio del flusso di controllo, etichettato con {true, false, uncond}
- Un GFC può essere costruito combinando opportunamente insieme i grafi relativi alle strutture di controllo

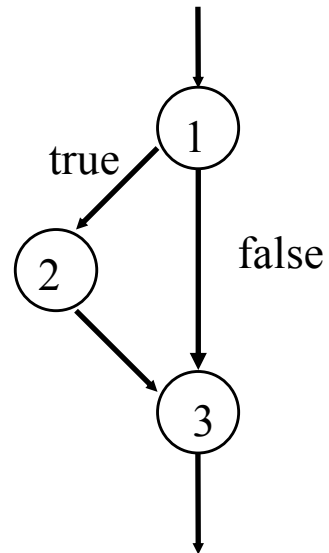


Costruzione del control flow graph per programmi strutturati

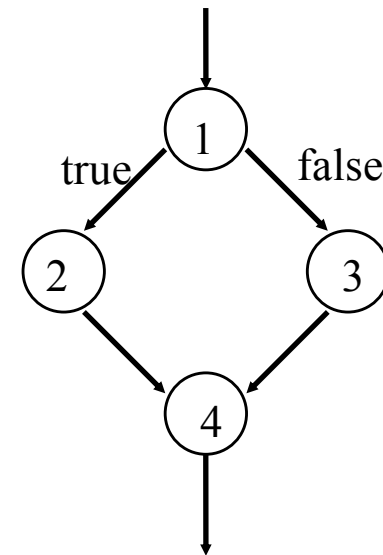
sequenza



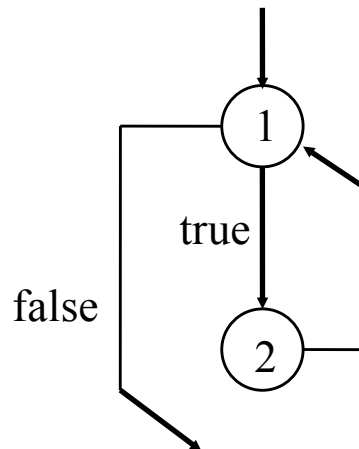
if



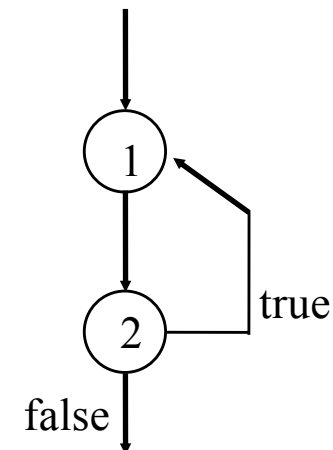
if else



while do

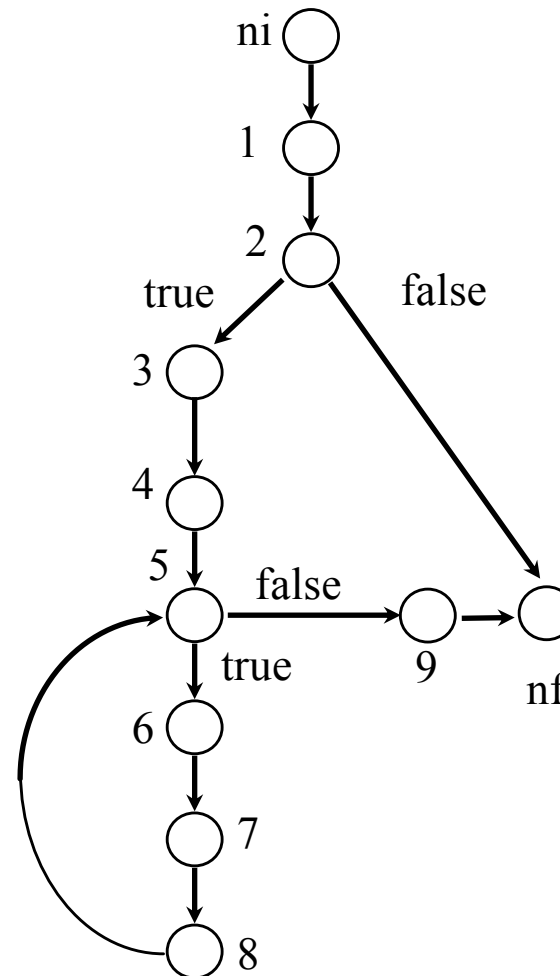


do ... while



Esempio di GFC

```
void Quadrato() {  
  int x, y, n;  
  1 cin >> x;  
  2 if (x > 0) {  
  3     n = 1;  
  4     y = 1;  
  5     while (x > 1) {  
  6         n = n + 2;  
  7         y = y + n;  
  8         x = x - 1;  
  9     }  
    cout << y;  
  }  
}
```

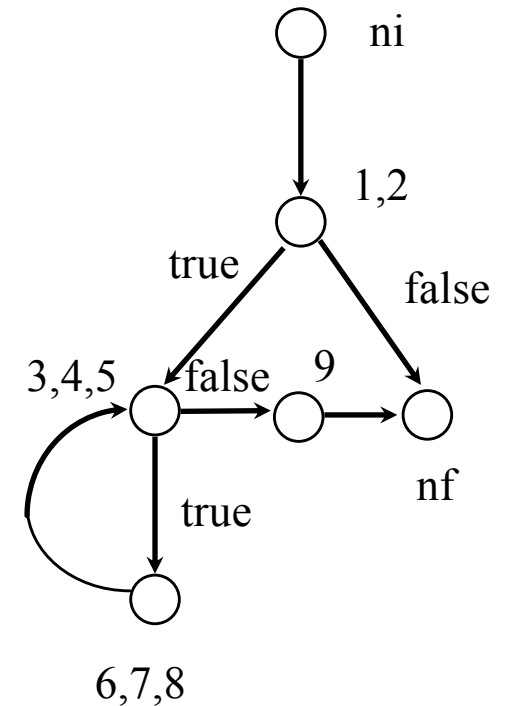
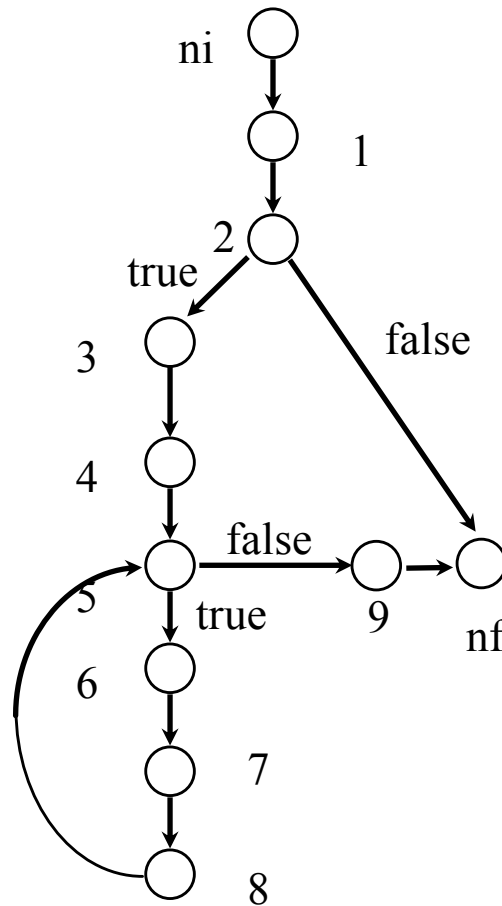


Semplificazione di un CFG

- ◆ Una sequenza di nodi può essere collassata in un solo nodo, purché nel grafo semplificato vengano mantenuti tutti i **branch** (*punti di decisione e biforcazione del flusso di controllo*)
- ◆ Tale nodo collassato può essere etichettato con i numeri dei nodi in esso ridotti

Esempio (con semplificazione)

```
void Quadrato() {  
  int x, y, n;  
  1 cin >> x;  
  2 if (x > 0) {  
  3     n = 1;  
  4     y = 1;  
  5     while (x > 1) {  
  6         n = n + 2;  
  7         y = y + n;  
  8         x = x - 1;  
    }  
  9     cout << y;  
  }
```



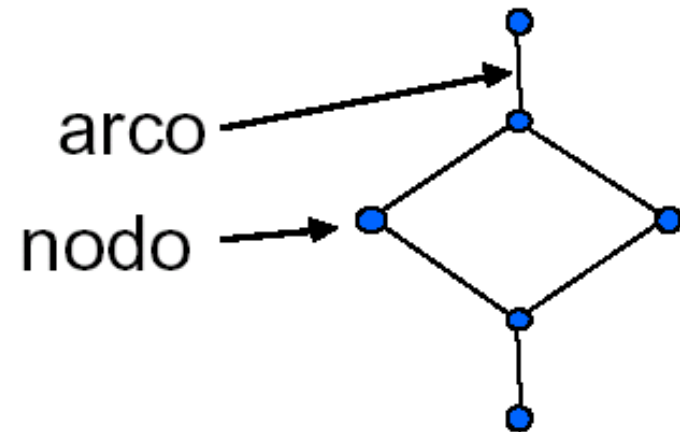
Cyclomatic complexity

- ◆ Metrica strutturale della sola *complessità del software* intesa in riferimento alla sua produzione, comprensione e modifica
- ◆ Cyclomatic complexity (CC)
 - Introdotta da Thomas McCabe nel 1976, è forse la metrica strutturale più usata
 - Misura il numero di cammini indipendenti attraverso un programma
- ◆ Cyclomatic number di un grafo G con n nodi, e archi e una componente connessa:

$$CC(G) = e - n + 2$$

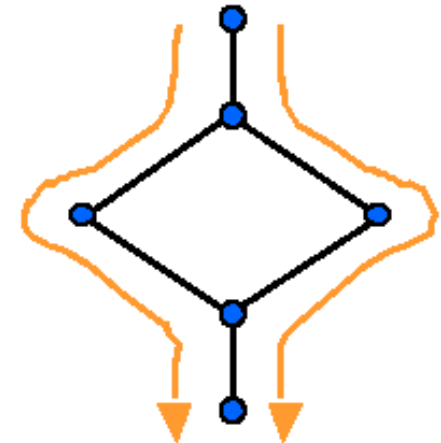
Cyclomatic Branch: calcolo

- ◆ Calcolato sul flowgraph del modulo
 - nodi (n) che rappresentano linee di codice
 - archi (e) rappresentano il flusso di controllo
- ◆ Esempio:
 - $n = 6$
 - $e = 6$
- ◆ La complessità ciclomatica è calcolata come $v(G)=e-n+2$.
- ◆ Per il grafo dell'esempio:
 - $V(G) = 6-6+2 = 2$



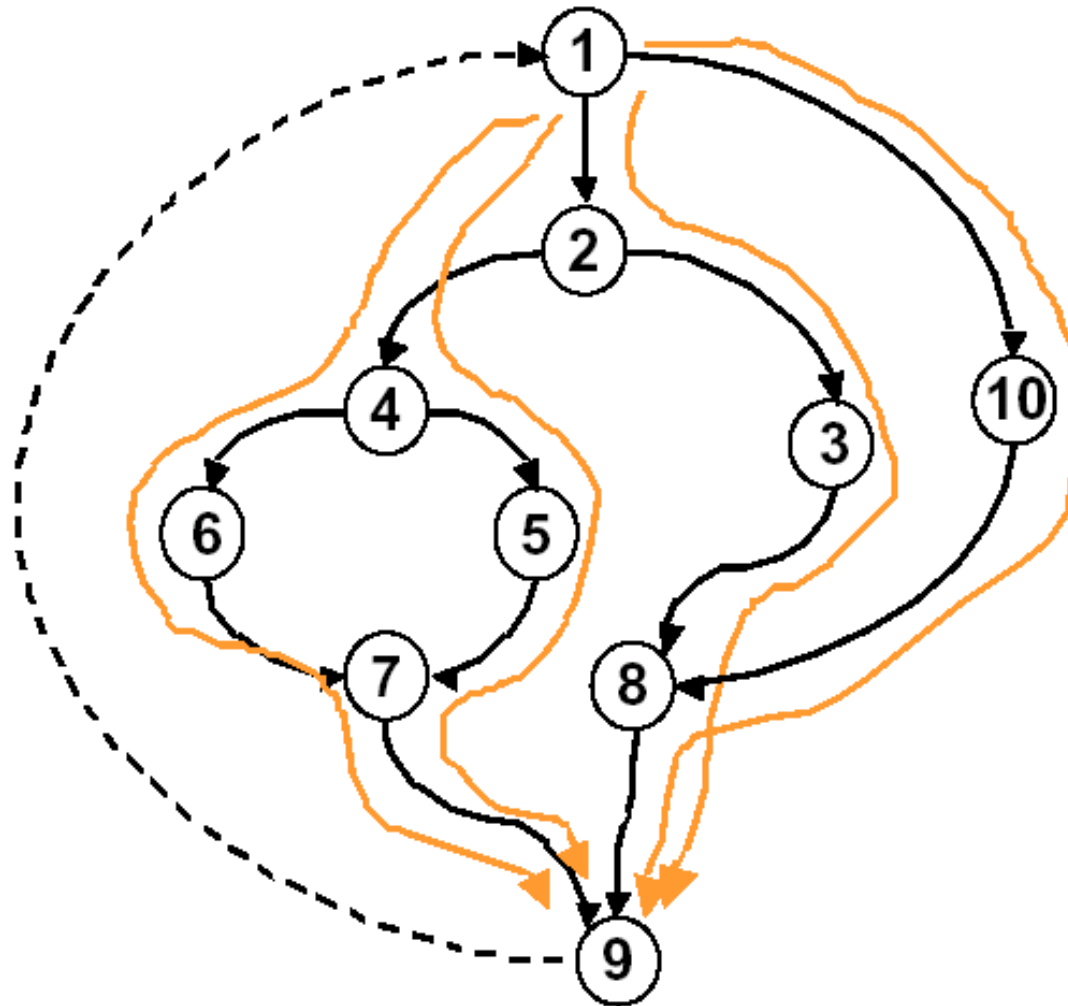
Cyclomatic Branch: significato

- ◆ Indica il numero di cammini indipendenti nel grafo (programma) dato.
- ◆ Quindi indica il limite superiore al numero di prove da effettuare per garantire che ogni arco sia eseguito almeno una volta.
- ◆ Un cammino è indipendente se introduce almeno un nuovo arco non incluso negli altri cammini.



Altro esempio di calcolo

- ◆ $e = 12$
- ◆ $n = 10$
- ◆ $V(G) = 4$



Utilizzo della complessità ciclomatica

- ◆ Un gran numero di programmi è stato misurato e analizzato.
- ◆ I risultati sono utilizzabili per stimare rischi, costi e stabilità dei programmi.
- ◆ Esiste correlazione tra cyclomatic complexity e frequenza degli errori.
- ◆ Complessità bassa indica anche buona manutenibilità e testabilità.

Cyclomatic complexity & test planning

- ◆ Le analisi hanno dimostrato che la cyclomatic complexity dà il numero esatto di test necessari per testare ogni punto di decisione di un programma per ogni risultato.
- ◆ Un modulo troppo complesso richiederà un numero di passi di test proibitivo.
 - In questo caso occorre suddividere il modulo in sotto-moduli meno complessi

Altri usi della complessità ciclomatica

- ◆ Code development risk analysis.
 - Valutazione del rischio durante la costruzione.
- ◆ Change risk analysis in manutenzione.
 - La complessità tende a crescere durante la manutenzione. Valutarla prima e dopo ogni intervento aiuta a tenere sotto controllo il rischio relativo.
- ◆ Reengineering.
 - Il rischio connesso con la reingegnerizzazione di un pezzo di codice è proporzionale alla complessità.

Valori di soglia

Cyclomatic
Complexity

Valutazione del rischio

1-10	programma semplice, rischio basso
11-20	più complesso, rischio moderato
21-50	complesso, alto rischio
> 50	programma non testabile, rischio estremamente alto

Critiche e avvertenze

- ◆ Rappresenta il numero di decisioni, più che la complessità vera e propria.
 - Manca il fondamento empirico della metrica.
- ◆ Comunque è comunemente accettato come indicatore di complessità.
 - Perché si correla bene con molti indicatori interessanti.
- ◆ In generale una complessità alta non è di per sé un problema: bisogna valutare anche la funzione del codice.

Comparison Cyclomatic

- ◆ Misura della complessità comparativa: è uguale alla somma delle istanze di confronto e di operatori logici (and, or, !=, ==, <=, >=, <, >) presenti nelle condizioni di test per ogni istruzione di test (if) e ciclo.
- ◆ Può essere usata per determinare il grado di complessità di percorsi di decisione.

Halstead Complexity Measures

- ◆ La misura della complessità di Halstead (1977) misura la complessità di un modulo direttamente dal codice sorgente (operandi e operatori), con enfasi sulla complessità computazionale.
- ◆ Usata spesso a supporto della manutenzione.
- ◆ Esistono opinioni molto diverse sulla sua validità.
- ◆ Sicuramente utile per valutare la qualità del codice in applicazioni dove il calcolo è predominante.

Halstead's Software Science

- ◆ A partire da pochi semplici elementi è in grado di misurare aspetti chiave del software
- ◆ La Software Science parte dall'assunto che ogni programma sia composto unicamente di *operatori ed operandi*
- ◆ Su questi sono definite delle *metriche*:
 - Indicatori definiti con lo scopo di misurare le caratteristiche di programmi *a posteriori*
 - Indicatori con intento *previsionale*

Halstead's Software Science

◆ Considera quattro elementi chiave:

- n_1 : number of distinct operators
- n_2 : number of distinct operands
- N_1 : total number of operators
- N_2 : total number of operands

```
if (k < 2)
{
    if (k > 3)
        x = x*k;
}
```

- Distinct operators:
 - *if* () { } > < = * ;
- Distinct operands:
 - *k* 2 3 *x*
- $n_1 = 10$
- $n_2 = 4$
- $N_1 = 13$
- $N_2 = 7$

Halstead's Metrics

- ◆ **Program length** (numero di elementi):

$$N = N_1 + N_2 \quad (13 + 7 = 20)$$

- ◆ **Program vocabulary** (numero di elementi distinti):

$$n = n_1 + n_2 \quad (10 + 4 = 14)$$

- ◆ **Program Volume:** $V = N \log_2 n \quad (20 \log_2 14 = 76.14)$

Program Volume

- ◆ Volume: $V = N \log_2 n$
- ◆ The Halstead Volume is based on the **Length** and the **Vocabulary**
 - *How much information does the reader of the code have to absorb to understand its meaning?*
- ◆ The biggest influence on the Volume metric is the Halstead length which causes a linear increase in the Volume (i.e., doubling the Length will double the Volume)
 - In the case of the Vocabulary the increase is logarithmic.
 - For example with a Length of 10 and a Vocabulary of 16 the Volume is 40. If we double the Length the Volume doubles to 80. If we keep the Length at 10 and double the Vocabulary to 32 we get a volume of 50

Le misure di Halstead

◆ Program length (N):

$$N = N_1 + N_2$$

◆ Program vocabulary (n):

$$n = n_1 + n_2$$

◆ Volume (V):

$$V = N * (\text{LOG}_2 n)$$

◆ Difficulty (D):

$$D = (n_1/2) * (N_2/n_2)$$

◆ Effort (E):

$$E = D * V$$

Critiche

- ◆ Misurano la complessità lessicale e/o testuale piuttosto che strutturale (come fa invece la cyclomatic complexity).
 - Comunque danno buone indicazioni, soprattutto in manutenzione.
- ◆ La complessità del codice con un alto rapporto di calcolo rispetto a controllo è valutata più accuratamente dalle misure di Halstead che non da cyclomatic complexity.

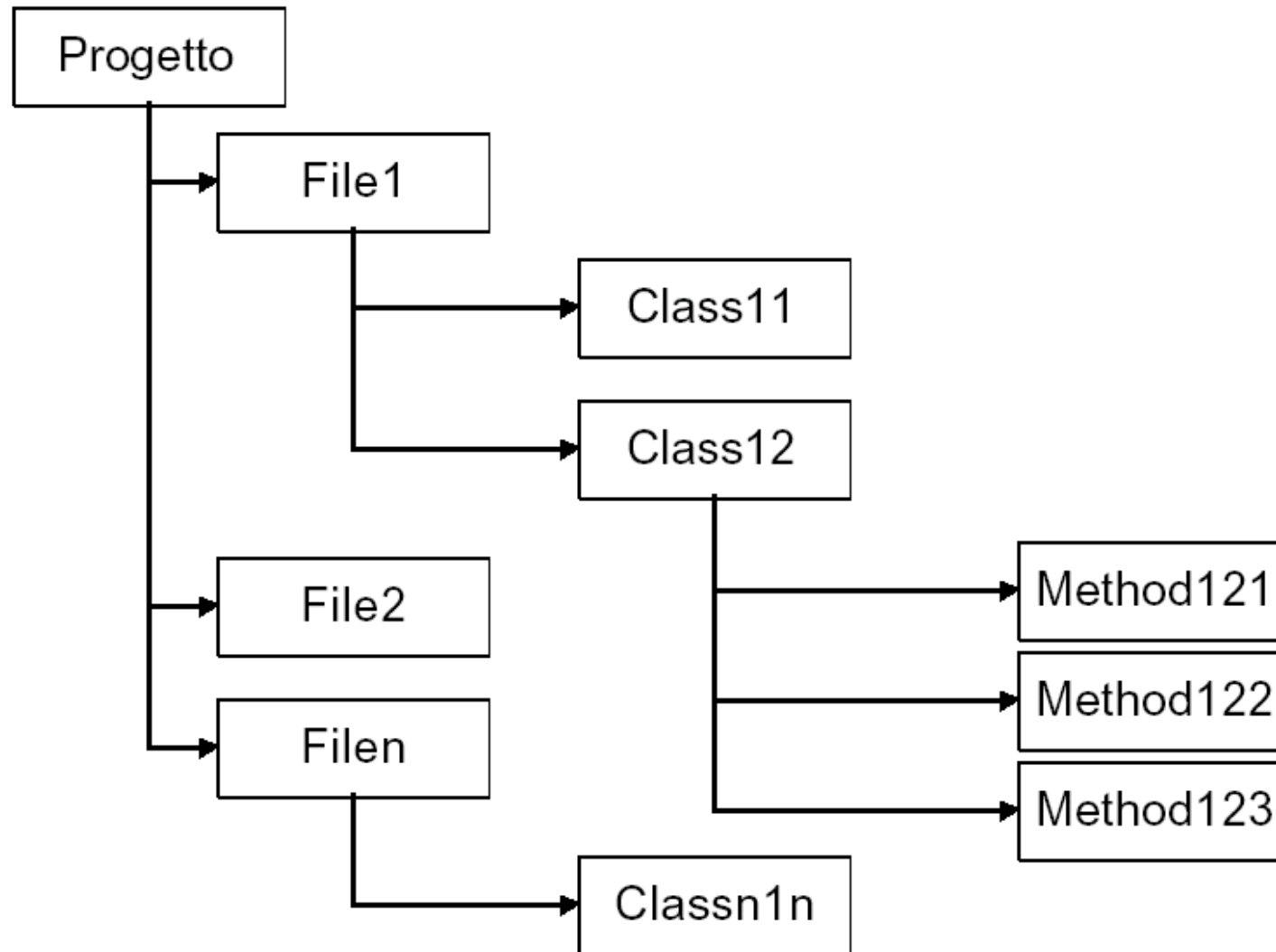
Altre metriche di complessità

- ◆ Per una descrizione e un confronto di tutte le metriche di complessità note in letteratura, si veda:
 - Marciniak, John J., ed. Encyclopedia of Software Engineering, 131-165. New York, NY: John Wiley & Sons, 1994.

Metriche per software object-oriented

- ◆ I sistemi software object-oriented sono strutturati in classi che contengono una parte pubblica e una parte privata.
- ◆ Ha quindi senso definire metriche relative alla strutturazione del sistema, e riportare le metriche classiche (LOC, complexity, ecc.) ai singoli elementi privati di ciascuna classe.

Struttura statica dei sistemi O.O.



Esempio di metriche statiche O.O.

- ◆ Metriche per Progetto:
 - ProjectTotalLines
 - ProjectLinesCode
 - FileCount
 - ProjectCommentLines
- ◆ Metriche per File:
 - FileTotalLines
 - FileLinesCode
 - ClassCount
 - FileCommentLines
- ◆ Metriche per Classe:
 - ◆ ClassTotalLines
 - ◆ MethodCount
- ◆ Metriche per Metodo:
 - ◆ MethodTotalLines
 - ◆ CyclomaticComplexity
 - ◆ InputParameter

CK Metrics: Objective

- ◆ Le metriche Chidamber and Kemerer (CK) sono state ideate per misurare aspetti peculiari del mondo O-O (es.: Incapsulamento, Information hiding, Inheritance, Abstraction):
 - Line of Code per Class (LOCC)
 - Weighted Methods per Class (WMC)
 - Depth of Inheritance Tree (DIT)
 - Number of children (NOC)
 - Coupling between object classes (CBO)
 - Response for class (RFC)
 - Lack of cohesion metric (LCOM)

CK Metrics: Objective

- ◆ Managers can improve the development of the SW by:
 - Analysing CK metrics through the identification of outlying values (extreme deviations), which may be a signal of:
 - » high complexity and/or
 - » possible design violations
 - Taking managerial decisions, such as:
 - » Re-designing and/or assigning extra or higher skilled resources (to develop, to test and to maintain the SW)

Line of code per class (LOCC)

$$LOCC(c) = \sum_{m \in M_{Impl}(c)} LOC(m)$$

- ◆ c : classe di cui si sta valutando LOCC (ignorando linee bianche o con soli commenti)
- ◆ $LOC(m)$: numero di linee di codice del metodo m
- ◆ $M_{Impl}(c)$: insieme dei metodi implementati nella classe c

Weighted methods per class (WMC)

- **Rationale:** The number of methods and complexity of methods involved is a direct predictor of how much time and effort is required to develop and maintain the class

$$WMC = \sum_{i=1}^n c_i$$

- ◆ c_i is the *complexity* of each method M_i of the class
 - Often, only public methods are considered
- ◆ Complexity may be the McCabe complexity of the method
- ◆ Smaller values are better
- ◆ **Objective: Low**

Weighted Methods per Class (WMC)

- ◆ Number of methods weighted by their procedural complexity (~complexity of class). Using unity weights gives simply number of methods in a class
- ◆ WMC is indicator of the amount of effort required to implement and test a class
 - As the number of methods for a class grows, it is likely to become more application specific and thus limiting possibilities for reuse
- ◆ High WMC value means also greater potential impact on children
- ◆ Difficult to set exact limits for metric, but WMC should be kept as low as possible
 - High WMC values -> split class

Depth of Inheritance Tree (DIT)

- ◆ For the system under examination, consider the hierarchy of classes
- ◆ DIT is the *length of the maximum path from the node to the root of the tree*
- ◆ Relates to the scope of the properties
 - How many ancestor classes can potentially affect a class
- ◆ As DIT grows, it is likely that classes on lower level inherit lots of methods and overrides some. Thus predicting behavior for an object of a class becomes difficult
- ◆ Large DIT means greater design complexity
 - DIT also measures reuse via inheritance
- ◆ High or low values of DIT might indicate problems with domain analysis
- ◆ In general, smaller values are better
- ◆ **Objective: Trade-off**

Number Of Children (NOC)

- ◆ For any class in the inheritance tree, NOC is the number of *immediate* children of the class
 - The number of direct subclasses
- ◆ How would you interpret this number?
- ◆ The greater the NOC is:
 - the greater is the RE-USE
 - the greater is the probability of improper abstraction of the parent class,
 - the greater the requirements of method's TESTING in that class
- Small values of NOC, may be an indicator of lack of communication between different class designers
- ◆ **Objective: Trade-off**
- ◆ **A moderate value indicates scope for reuse and high values may indicate an inappropriate abstraction in the design**

Number Of Children (NOC)

- ◆ Indicator of potential influence that class has on design
- ◆ High value of NOC might indicate misuse of subclassing (*i.e., implementation inheritance instead of is-a relationship*)
 - NOC is also measure of reuse via inheritance
- ◆ Class with very high NOC might be a candidate for refactoring to create more maintainable hierarchy
- ◆ NOC can be used as indicator of required testing time

Coupling Between Object classes (CBO)

- ◆ Also known as: **Fan-Out**
- ◆ For a class **C**, the CBO metric is a count of the number of other classes to which the class **C** is coupled
- ◆ A class **X**, is coupled to class **C** if
 - **X** operates on (affects) **C** or
 - **C** operates on **X**
- ◆ Excessive coupling indicates weakness of class encapsulation and may inhibit reuse
- ◆ High coupling also indicates that more faults may be introduced due to inter-class activities

Coupling Between Object classes (CBO)

◆ Small values of CBO :

- Improve MODULARITY and promote ENCAPSULATION
- Indicates independence in the class, making easier its RE-USE
- Makes easier to MAINTAIN and to TEST a class

◆ **Objective: Low**

Response For Class (RFC)

- It is the number of methods of the class plus the number of methods called by any of those methods
 - $RFC(c) = \# \text{ methods of } c + \# \text{ methods of other classes invoked by } c \text{ (recursively)}$
- If a large number of methods can be invoked in response to a message, the testing and debugging of the class becomes more complicated

$$RFC = \sum_{i=1}^n Mc_i$$

- ◆ Mc_i # of methods called in response to a message that invokes method M_i
 - Fully nested set of calls

Response For Class (RFC)

◆ Viewpoints

- If a large numbers of methods are invoked from a class (RFC is high):
 - » TESTING and MAINTANACE of the Class becomes more COMPLEX

◆ Objective: Low

Lack of COhesion Metric (LCOM)

- ◆ Measures the dissimilarity of methods in a class via instance variables
- ◆ Per ogni variabile della classe, si calcola la percentuale di metodi che ne fanno uso. Si calcola la media e si sottrae dal 100%
- ◆ *L'idea è che se più metodi usano una stessa variabile di istanza, allora questi metodi fanno cose “simili”, cioè sono coesi*
- ◆ A measure of the “tightness” of the code
- ◆ If a method references many instance variables, then it is more complex, and less cohesive
- ◆ The larger the number of similar methods in a class the more cohesive the class is
- ◆ Cohesiveness of methods within a class is desirable, since it promotes encapsulation

Lack of COhesion Metric (LCOM)

◆ Great values of LCOM:

- Increases COMPLEXITY
- Does not promotes ENCAPSULATION and implies classes should probably be split into two or more subclasses
- Helps to identified low-quality design

◆ Objective: Low

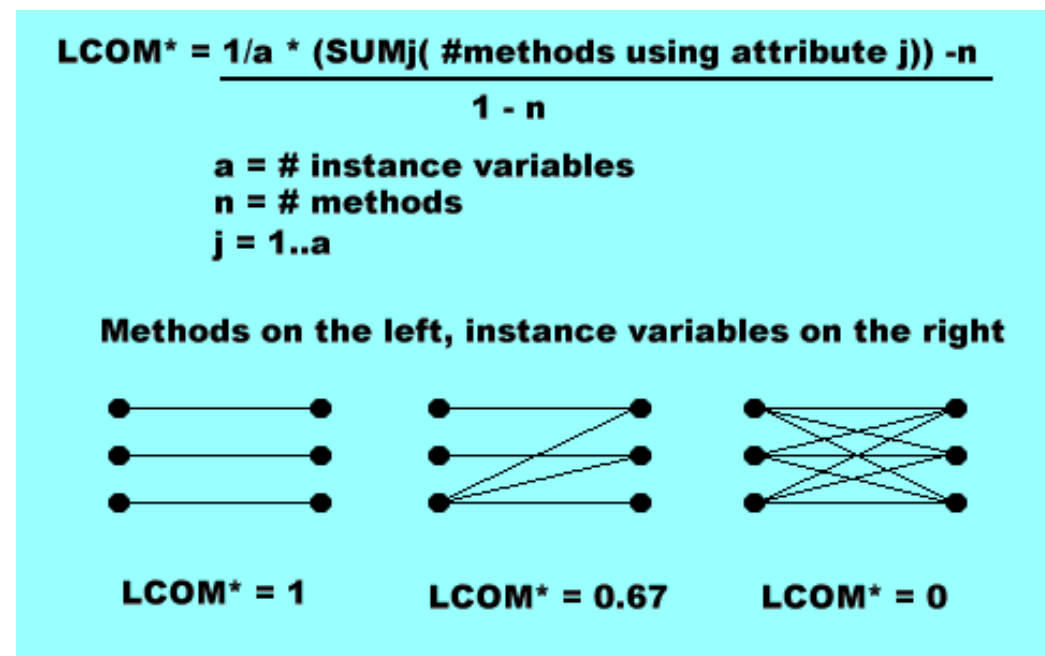
Lack of COhesion in Methods (LCOM)

- ◆ LCOM counts the sets of methods that are not related through the sharing of some of the class's instance variables
 - LCOM* normalized version, range of values between 0..1
 - LCOM* = 0 if every method uses all instance variables
 - LCOM* = 1 if every method uses only one instance variable
- ◆ High values of LCOM indicate scatter in the functionality provided by a class, i.e., class attempts to provide many different objectives
- ◆ Classes with high LCOM can be fault-prone and are likely to behave less predictable ways than classes with low LCOM
- ◆ High LCOM -> split class

Lack of COhesion in Methods (LCOM)

◆ Problems with LCOM:

- Classes with "getters & setters" (getProperty(), setProperty()) get high LCOM values although this is not an indication of a problem
- Don't even try to measure logical cohesion



CK Metrics: Guidelines

◆ *But How much is Low and High ?*

METRIC	GOAL	LEVEL	COMPLEXITY (To develop, to test and to maintain)	RE-USABILITY	ENCAPSULATION, MODULARITY
WMC	Low	▼	▼	▲	
DIT	Trade-off	▼	▼	▼	
		▲	▲	▲	
NOC	Trade-off	▼	▼	▼	
		▲	▲	▲	
CBO	Low	▼	▼		▲
RFC	Low	▼	▼		
LCOM	Low	▼	▼		▲

CK Metrics: Thresholds

- ◆ Thresholds of the CK metrics:
 - Can not be determined before their use
 - Should be derived and use locally for each dataset
 - 80th and 20th percentiles of the distributions can be used to determine high and low values of the metrics
 - Are not indicators of “badness” but indicators of difference that needs to be investigated

Strumenti di misurazione commerciali

- ◆ Cantata
- ◆ CMT++
- ◆ Codecheck
- ◆ Ensemble
- ◆ ENVY/QA
- ◆ Hindsight
- ◆ Krakatau
- ◆ LDRA's Testbed
- ◆ Logiscope
- ◆ Metamata Metrics
- ◆ Metrics ONE
- ◆ Metrics4[FORTRAN | Pascal | Project]
- ◆ Object Detail
- ◆ OOMetric/Developer
- ◆ Panorama
- ◆ PCA-RCM
- ◆ PC-Metric
- ◆ QA Family
- ◆ QualGen
- ◆ Resource Standard Metrics
- ◆ Software Metrics
- ◆ Testworks METRIC
- ◆ Total Metric for Java
- ◆ VISION:Assess
- ◆ Zinnote

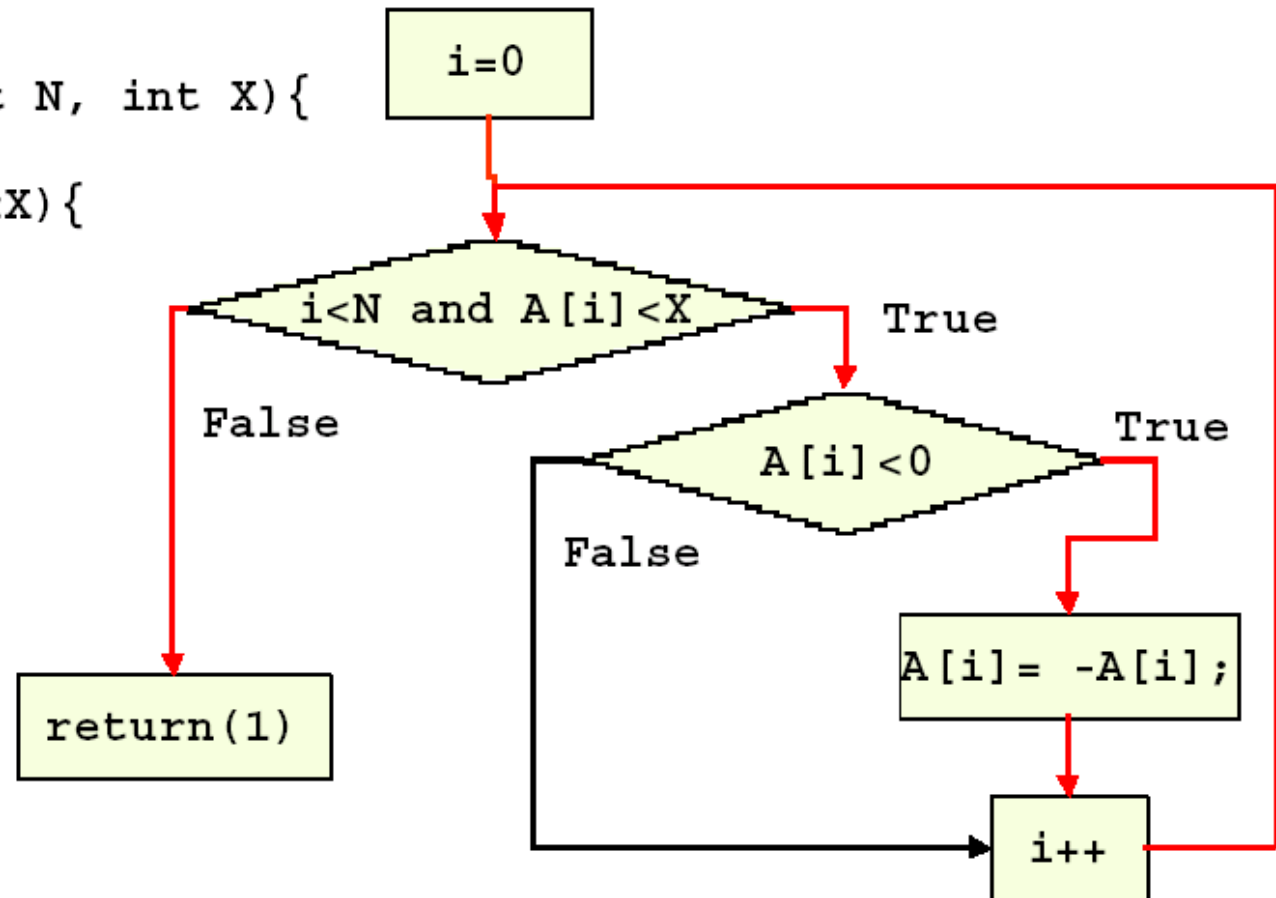
Metriche dinamiche

Copertura strutturale

- ◆ criteri di (in)adeguatezza
 - se parti significative del programma non sono testate, il test è sicuramente inadeguato
- ◆ criteri di copertura del flusso di controllo
 - copertura delle istruzioni
 - copertura delle decisioni
 - copertura delle condizioni
 - copertura dei cammini
 - copertura del flusso dei dati
- ◆ tentativo di compromesso tra impossibile ed inadeguato

Copertura delle istruzioni

```
int select(int A[], int N, int X){  
    int i=0;  
    while (i<N and A[i] <X){  
        if (A[i]<0)  
            A[i] = - A[i];  
        i++;  
    }  
    return(1);  
}
```

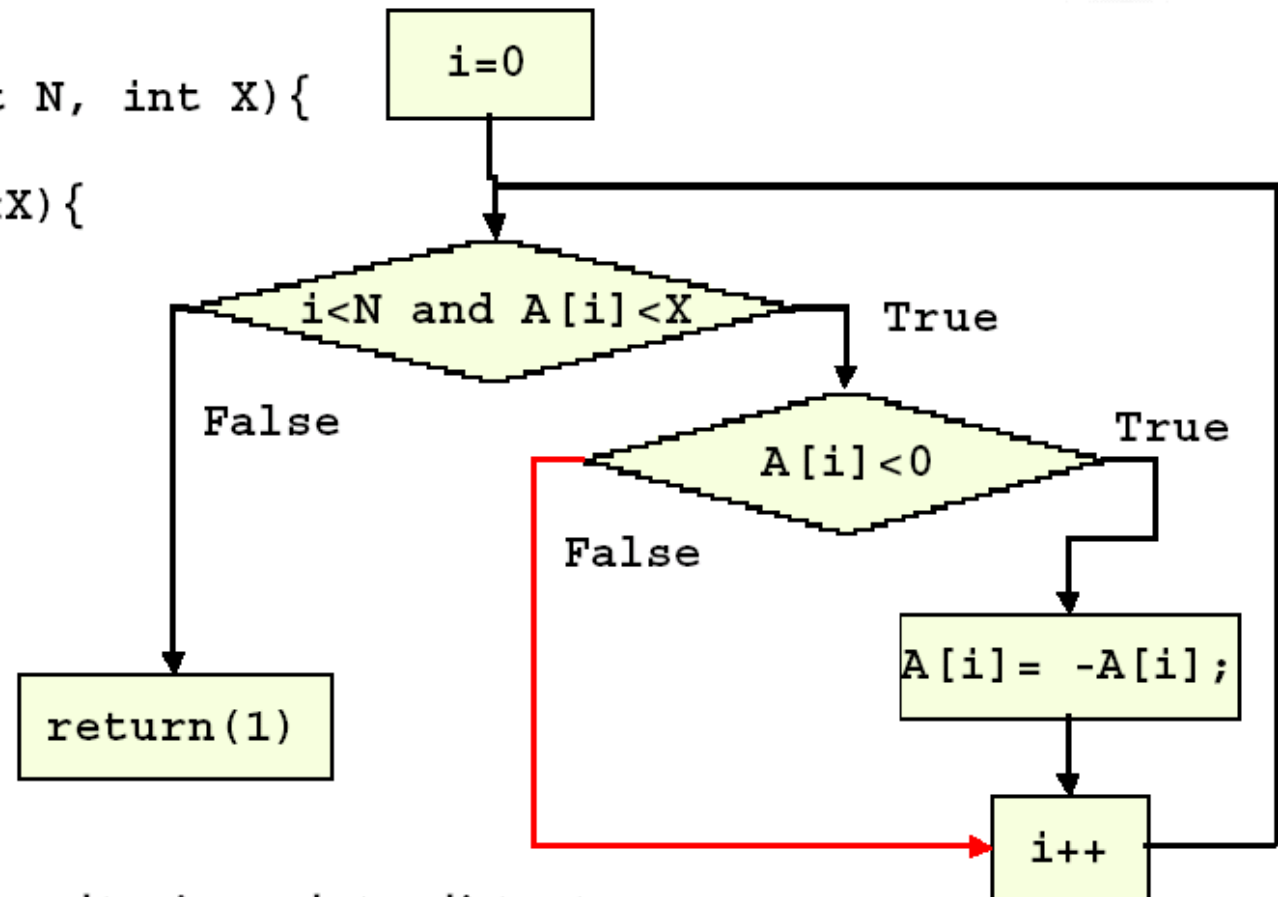


Un solo dato di test ($N=1$, $A[0]=-7$, $X=9$) è sufficiente per garantire la copertura delle istruzioni ...

Ma errori nel trattare valori positivi di $A[i]$ non sono rilevabili

Copertura delle decisioni

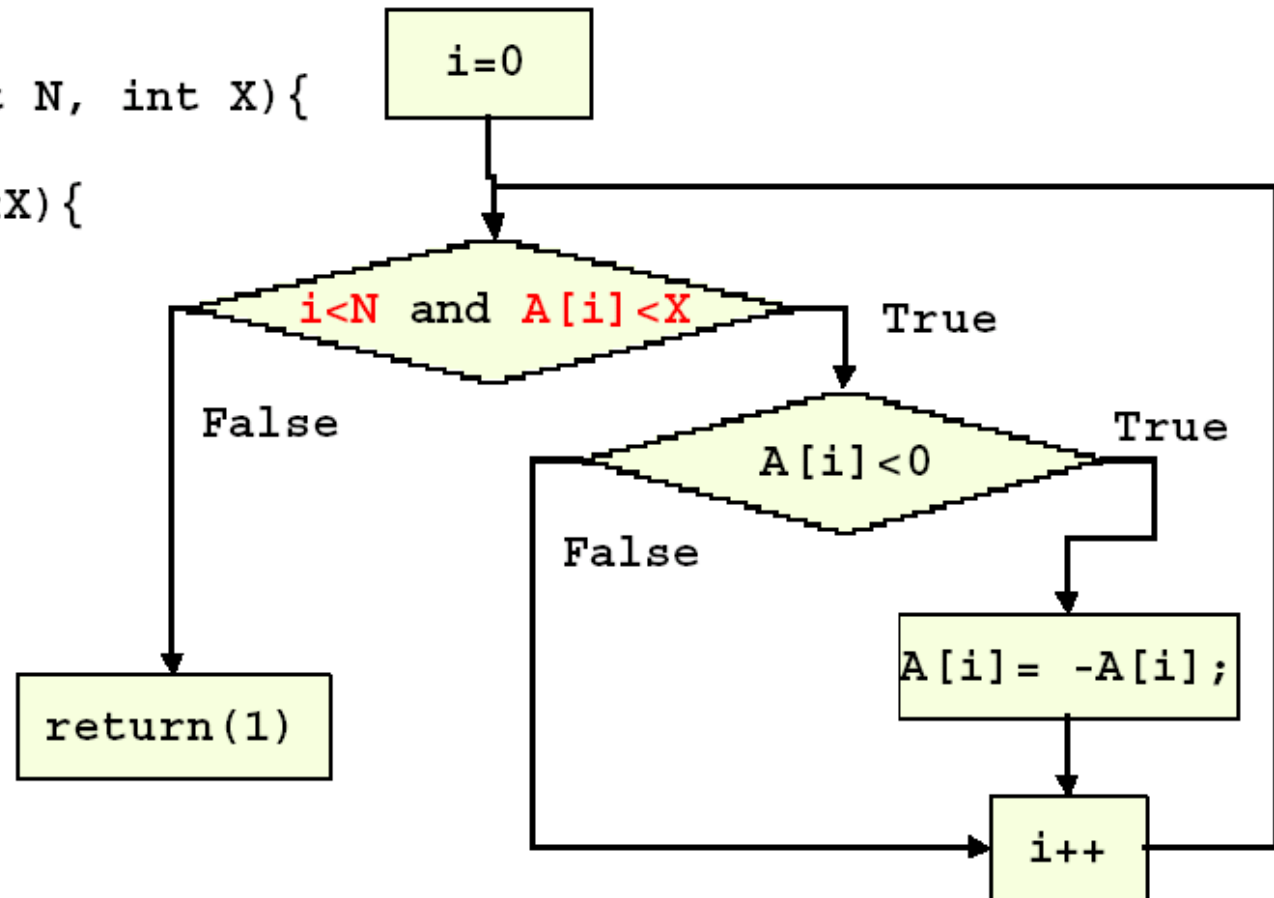
```
int select(int A[], int N, int X){  
    int i=0;  
    while (i<N and A[i] <X){  
        if (A[i]<0)  
            A[i] = - A[i];  
        i++;  
    }  
    return(1);  
}
```



Dobbiamo aggiungere un ulteriore dato di test
($N=1$, $A[0]=7$, $X=9$) per coprire il ramo *falso* del comando `if`.
Errori nel trattare valori positivi di $A[i]$ sarebbero rilevabili. Errori nell'uscita dal ciclo con condizione $A[i] < X$ non sarebbero rilevati

Copertura delle condizioni base

```
int select(int A[], int N, int X){  
    int i=0;  
    while (i<N and A[i] <X){  
        if (A[i]<0)  
            A[i] = - A[i];  
        i++;  
    }  
    return(1);  
}
```

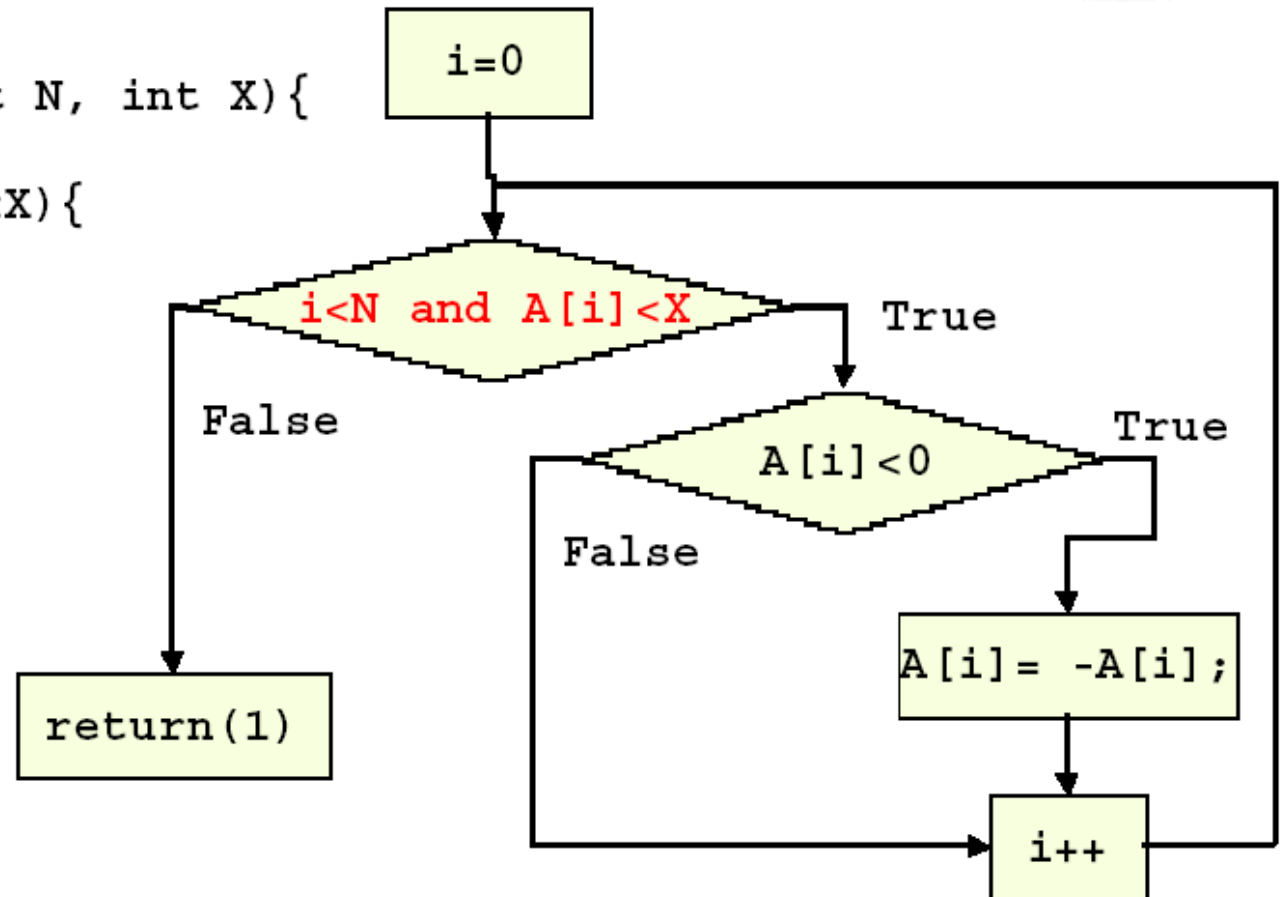


Entrambe le condizioni ($i < N$), ($A[i] < X$) devono valere *falso* e *vero* per test differenti.

In questo caso dobbiamo aggiungere casi di test che fanno terminare il ciclo con la condizione $\text{not}(A[i] < X)$.

Copertura delle condizioni composte

```
int select(int A[], int N, int X){  
    int i=0;  
    while (i<N and A[i] <X){  
        if (A[i]<0)  
            A[i] = - A[i];  
        i++;  
    }  
    return(1);  
}
```



Test di tutte le combinazioni di valori di verità delle condizioni.
Potrebbe non essere fattibile.

Copertura delle condizioni modificate

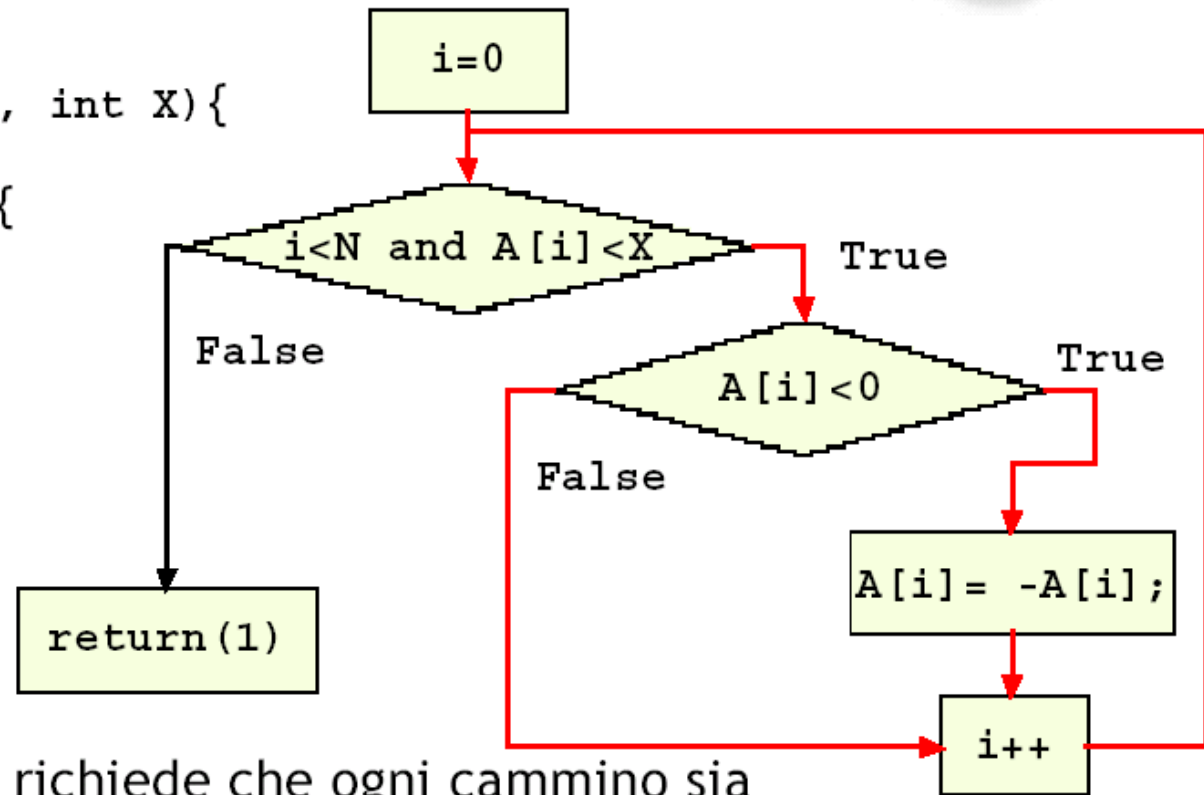
- ◆ Il valore di ogni condizione base deve determinare il valore di verità della decisione

```
if (A or (B and C))  
  then do_something  
  else do_someth_else  
end if;
```

#	A	B	C	Out
1	false	false	true	false
2	true	false	true	true
3	false	true	true	true
4	false	true	false	false

Copertura dei cammini

```
int select(int A[], int N, int X){  
    int i=0;  
    while (i<N and A[i] <X){  
        if (A[i]<0)  
            A[i] = - A[i];  
        i++;  
    }  
    return(1);  
}
```



La copertura dei cammini richiede che ogni cammino sia eseguito almeno una volta, rivelando così i problemi che si verificano quando i cicli sono iterati diverse volte.

Problema: crescita incontrollata dei dati di test: il programma in figura ha infiniti cammini possibili. Dobbiamo selezionare un sottoinsieme ragionevole di casi, riducendo le iterazioni.

Il problema della non eseguibilità

- ◆ Comportamenti sintatticamente validi (cammini, flussi di dati,...) possono essere non eseguibili
- ◆ I criteri di adeguatezza sono spesso impossibili da soddisfare.
- ◆ Approcci possibili:
 - giustificazione manuale dell'omissione di casi di test
 - accettazione di copertura parziale

Metriche e processo

- ◆ Come bilanciare budget, pianificazione, rischi e qualità
 - Necessità di
 - » ridurre budget
 - » garantire qualità
 - » minimizzare rischi
 - » anticipare costi (pianificare)
 - Non esistono regole generali
 - Si possono realizzare meccanismi per ambienti specifici



Prima fase: controllo della fase di test

◆ Raccolta dati storici

- misure statiche su codice (LOC, numero ciclomatico, software science,...)
- misure dinamiche su test effettuati (copertura codice e specifiche)
- dati statistici su errori:
 - » localizzazione (modulo, blocco)
 - » tipo (violazione di memoria, errore passaggio parametri,...)
 - » gravità (catastrofico, ..., cosmetico))

◆ Elaborazione statistica dei dati

- costruzione modelli

Modelli statistici

◆ Modelli:

- relazione statistica tra metriche statiche e
 - » presenza errori (per classi di errori)
 - » numero errori (per classi di errori)
- relazione statistica tra metriche statiche, metriche dinamiche e
 - » presenza errori (per classi di errori)
 - » numero errori (per classi di errori)

◆ Possibile predire distribuzione errori per modulo

Esempio di Modello

- ◆ Insieme di metriche “sensibili” rispetto a presenza di errori
- ◆ Exits = numero di uscite
- ◆ Intervals = numero di blocchi
- ◆ C-head = numero di commenti in testa alle funzioni
- ◆ Lines = numero di linee di codice

Uso di modelli statistici per controllo budget

- ◆ Modelli statistici che fanno riferimento a metriche statiche permettono di identificare moduli particolarmente “pericolosi”:
 - può essere più conveniente scartare il modulo e richiedere nuova produzione
 - può essere necessario ridisegnare il budget (o un diverso livello di qualità richiesto) per la fase di test prima dell’inizio del testing => possibilità di anticipare interventi e quindi ottenere risultati migliori

Uso di modelli statistici per pianificazione

- ◆ Piano di test standard per tutti i moduli
- ◆ Piano di test eccezionale per moduli “pericolosi”
- ◆ Pianificazione test in funzione di
 - rischio del modulo
 - qualità richiesta
 - potenziale difettosità del modulo

Uso di modelli di test per controllo di rischi e qualità

- ◆ Modelli statistici che fanno riferimento a metriche statiche e dinamiche permettono di identificare moduli particolarmente a rischio dopo la fase di test == alta probabilità di difetti presenti dedotti da complessità del modulo e copertura ottenuta:
 - possibile identificare moduli che
 - » richiedono ulteriore test
 - » possono causare problemi in fase di installazione e/o uso

Introduzione nel processo di test

◆ Aspetti tecnici

- necessari strumenti di raccolta di metriche statiche e dinamiche
- necessaria automazione di raccolta ed elaborazione

◆ Aspetti psicologici

- controllare impatto su progettisti
 - » chiarire scopi e metriche utilizzate
- controllare impatto su addetti al test
 - » i risultati hanno solo valenza statistica => tutti i componenti vanno testati

Meccanismo di messa a punto come condizione essenziale

- ◆ I modelli hanno validità statistica
- ◆ i modelli sono fortemente dipendenti dal programma analizzato
- ◆ la messa a punto del modello su un numero maggiore di dati permette di avere valori statistici più stabili
- ◆ la revisione del modello su nuovi dati permette di
 - adattarsi a nuovi tipi di software
 - mantenere consistenza con evoluzione del team di sviluppo (feedback)

Implementazione di un meccanismo di messa a punto

- ◆ Raccolta di un ampio insieme di metriche statiche e dinamiche
- ◆ raccolta delle informazioni necessarie sugli errori rilevati
 - tipo
 - frequenza
 - gravità
 - localizzazioni
- ◆ Costruzione periodica di nuovi modelli statistici e confronto con i modelli precedenti

Complementarità con tecniche di ispezione

- ◆ Anomalie del codice possono influenzare in modo scorretto il codice
 - esempi: distribuzione linee bianche, numero linee di commento,...
- ◆ Alcune caratteristiche del codice non si riflettono nelle metriche
 - esempi: aritmetica dei puntatori,...
- ◆ Tecniche di ispezione di codice possono rilevare ed eliminare anomalie fastidiose e rendere più precisi i risultati